

BINX TECH • BACKEND DEVELOPMENT INTERNSHIP PROGRAM (.NET)

WEEK 2

ADVANCED C# & ASP.NET CORE FOUNDATIONS

Going deeper into C#'s type system and concurrency model, then building the first ASP.NET Core Web API from scratch.

Week 2 carries Phase 1 into Phase 2. Interns extend last week's C# fundamentals with generics, deeper LINQ, and a more rigorous async/await model, then set up their first ASP.NET Core Web API project — routing, the middleware pipeline, and dependency injection — the exact request/response path every endpoint built for the rest of this program will run through.

PHASE 1 → 2 • 40 HOURS • 5 TRAINING DAYS • ASP.NET CORE FOUNDATIONS

Prepared by BinX Tech

Palestine | Nablus

BinX
TECH



1. Week 2 Overview

Field	Detail
Week	Week 2 of 10 — Phase 1 → 2: Advanced C# & ASP.NET Core Foundations
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 3-4 combined)
Format	On-site / Remote / Hybrid — API project committed to GitHub, tested via Postman
Focus	Advanced C# (LINQ, async/await, generics); begin ASP.NET Core routing and middleware
Prerequisite	Week 1 completed — working .NET environment, C# fundamentals, Git workflow
Mentor Supervision	Async/concurrency check-in on Day 3; ASP.NET Core project review on Day 5

2. Week 2 Learning Objectives

- Write generic methods and classes with appropriate constraints.
- Use advanced LINQ operations: grouping, joining, and flattening nested collections.
- Apply the Task-based asynchronous pattern correctly, avoiding common async pitfalls.
- Scaffold an ASP.NET Core Web API project and define routes using Controllers and Minimal APIs.
- Understand and configure the middleware pipeline and its execution order.
- Register and inject services using ASP.NET Core's built-in dependency injection container.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Generics and advanced collections
Day 2	8 hrs	Advanced LINQ and deferred execution
Day 3	8 hrs	Async/await deep dive and concurrency basics
Day 4	8 hrs	ASP.NET Core project setup and routing
Day 5	8 hrs	Middleware pipeline and dependency injection; Week 2 synthesis

4. Day-by-Day Curriculum

Day 1 — Generics & Advanced Collections — 8 hours

Learning Objectives

- Explain why generics exist and what problem they solve over object-typed collections.
- Write generic methods and classes with type parameters.
- Apply generic constraints to restrict what a type parameter can be.

Key Topics

- Why generics
- Writing generic methods and classes
- Generic constraints
- Choosing between collection interfaces

Lesson Content

1.1 Why Generics

Before generics, a reusable collection had to store everything as “object,” losing type safety and forcing a cast on every read — and that cast could fail at runtime with no compile-time warning. A generic type like `List<T>` keeps full type safety while still being reusable across any type: `List<int>` and `List<string>` share one implementation but the compiler enforces exactly what each instance is allowed to hold.

1.2 Writing Generic Methods and Classes

A generic method or class declares a type parameter (conventionally “T”, or a descriptive name like “TEntity” for clarity) in angle brackets, which the compiler substitutes with a real type at each call site. This is the same mechanism behind `List<T>`, `Dictionary<TKey, TValue>`, and, from Week 3 onward, the generic repository and service patterns this program's ASP.NET Core projects will use throughout.

```
public class Repository<T> where T : class
{
    private readonly List<T> _items = new();
    public void Add(T item) => _items.Add(item);
    public IReadOnlyList<T> GetAll() => _items.AsReadOnly();
}
```

1.3 Generic Constraints

A constraint (“where T : class”, “where T : IComparable”, “where T : new()”) restricts what types can be substituted for a type parameter, letting the generic code call members that wouldn't otherwise be guaranteed to exist. Without a constraint, a generic method can only rely on the members every possible type shares (essentially just object's members); a constraint unlocks calling more specific behavior safely.

1.4 Choosing Between Collection Interfaces

`IEnumerable<T>` guarantees only forward iteration — the right return type for a method that just produces a sequence. `IReadOnlyList<T>` adds indexed access and a known count without allowing modification — a good public-facing return type when callers shouldn't be able to mutate what they receive. `IList<T>` allows full mutation and is the right choice only when the caller genuinely needs to add, remove, or replace items. Returning the least permissive interface that still satisfies the caller's need is a habit that pays off as an API surface grows.



Returning "List<T>" from every public method is a common shortcut that quietly leaks mutability the caller was never meant to have. Prefer IEnumerable<T> or IReadOnlyList<T> unless the caller genuinely needs to modify the collection.

Hands-On Lab: Build a Generic Repository

1. Write a generic Repository<T> class with Add, GetAll, and a Find method taking a predicate.
2. Apply a "where T : class" constraint and explain in a comment why it's needed.
3. Instantiate the repository with two different types from your Week 1 domain model.
4. Change GetAll's return type to IReadOnlyList<T> and confirm the caller can no longer modify the result directly.
5. Commit the generic repository to your GitHub repository.

Tools Used

.NET SDK • VS Code or Visual Studio

Day 2 — Advanced LINQ & Deferred Execution — 8 hours

Learning Objectives

- Explain the difference between deferred and immediate LINQ execution.
- Use GroupBy and Join to reshape and combine data.
- Flatten nested collections using SelectMany.

Key Topics

- Deferred vs. immediate execution
- Grouping and joining data
- Flattening with SelectMany
- Common LINQ performance pitfalls

Lesson Content

2.1 Deferred vs. Immediate Execution

A LINQ query built with Where, Select, or OrderBy doesn't actually run until it's enumerated — by a foreach loop, or by calling ToList(), ToArray(), or Count(). This deferred execution means the same query variable can silently return different results if the underlying data changes between when it's defined and when it's enumerated — a subtle source of bugs until the behavior is understood and expected.

2.2 Grouping and Joining Data

GroupBy clusters elements by a key, producing a sequence of groups each with their own key and members — useful for something like grouping orders by customer. Join combines two collections on a matching key, similar to a SQL inner join, useful when related data lives in two separate collections that need to be combined into one shape before further processing.

```
var ordersByCustomer = orders
    .GroupBy(o => o.CustomerId)
    .Select(g => new { CustomerId = g.Key, Total = g.Sum(o => o.Amount) });

var customerOrders = customers
    .Join(orders, c => c.Id, o => o.CustomerId,
        (c, o) => new { c.Name, o.Amount });
```



2.3 Flattening with SelectMany

Select applied to a collection of collections produces a collection of collections (a customer with a list of orders becomes a list of lists) — SelectMany flattens that one level, producing a single sequence of every order across every customer directly. This is the correct tool whenever a query needs to work with the inner items directly rather than the outer grouping structure.

2.4 Common LINQ Performance Pitfalls

Calling ToList() partway through a chain and continuing to query the result forces an eager materialization that can be wasteful if further filtering was going to reduce the set anyway. Re-enumerating the same deferred query multiple times (once for a Count(), again for a foreach) re-runs the entire query each time — materializing once with ToList() and reusing that in-memory list is often the more efficient choice when a query result is used more than once.

Deferred execution is a feature, not a bug — but only once its behavior is deliberate. If a query's timing relative to the data it queries matters, materialize it explicitly with ToList() at the exact point that matters.

Hands-On Lab: Grouping, Joining & Flattening

1. Create two related collections (e.g. Customers and Orders) with at least 6 items each, sharing a foreign key.
2. Write a GroupBy query summarizing total order amount per customer.
3. Write a Join query combining customer names with their order amounts.
4. Write a SelectMany query flattening a nested collection (e.g. every order line item across every order) into one sequence.
5. Demonstrate deferred execution by modifying the source collection after defining a query but before enumerating it, and explain the result.

Tools Used

.NET SDK • VS Code or Visual Studio

Day 3 — Async/Await Deep Dive & Concurrency Basics — 8 hours

Learning Objectives

- Explain the Task-based asynchronous pattern and how it differs from synchronous code.
- Avoid the most common async pitfalls, especially blocking on async code.
- Run independent operations concurrently using Task.WhenAll.

Key Topics

- The Task-based asynchronous pattern
- Async all the way — avoiding common pitfalls
- Running operations concurrently with Task.WhenAll
- Cancellation tokens

Lesson Content

3.1 The Task-Based Asynchronous Pattern

A Task represents an operation that may not have completed yet; Task<T> represents the same thing but with a result value once it does complete. Awaiting a Task doesn't block the calling thread — it schedules the rest of the method to resume once the awaited operation finishes, freeing the thread to do other work in the meantime. This is what lets a single ASP.NET Core server handle thousands of concurrent requests without needing thousands of dedicated threads.



3.2 Async All the Way

Calling “.Result” or “.Wait()” on a Task from synchronous code forces the calling thread to block until the async operation completes, defeating the entire purpose of using async in the first place — and in some hosting contexts, it can cause an actual deadlock. The rule this program enforces is “async all the way”: once a call chain goes async at any point, every caller up the chain should also be async, using await consistently rather than blocking.

```
// Avoid: blocks the calling thread, risks deadlock
var result = GetDataAsync().Result;

// Correct: async all the way up the call chain
var result = await GetDataAsync();
```

3.3 Running Operations Concurrently with Task.WhenAll

Awaiting three independent async calls one after another runs them sequentially, each waiting for the last to finish even though nothing requires that order. Starting all three tasks first, then awaiting them together with Task.WhenAll, lets them run concurrently — the total time becomes roughly the slowest single operation instead of the sum of all three, a meaningful difference once real network or database latency is involved.

3.4 Cancellation Tokens

A CancellationToken, passed through an async call chain, lets a long-running operation be told to stop early — essential for an API where a client disconnects or a request times out and the server shouldn't keep doing wasted work on their behalf. ASP.NET Core automatically provides a cancellation token tied to the HTTP request's lifetime, which this program's Week 3 API endpoints will start accepting and honoring.

A method named “GetDataAsync” that internally calls “.Result” on another task isn't actually async — it's a synchronous method wearing an async name. The naming convention is a promise about behavior, not just a suffix.

Hands-On Lab: Concurrent Async Operations

1. Write 3 async methods that each simulate a delay (Task.Delay) representing a different data source.
2. Call all 3 sequentially with individual awaits and measure the total elapsed time.
3. Rewrite the same calls using Task.WhenAll and compare the elapsed time.
4. Add a CancellationToken parameter to one method and demonstrate cancelling it mid-operation.
5. Commit the concurrency demo to your GitHub repository.

Tools Used

.NET SDK • VS Code or Visual Studio



Day 4 — ASP.NET Core Project Setup & Routing — 8 hours

Learning Objectives

- Scaffold a new ASP.NET Core Web API project using the dotnet CLI.
- Understand the minimal hosting model in Program.cs.
- Define routes using both Controllers and Minimal APIs.

Key Topics

- Scaffolding a Web API project
- The minimal hosting model (Program.cs)
- Controllers vs. Minimal APIs
- Routes, route parameters, and HTTP verbs

Lesson Content

4.1 Scaffolding a Web API Project

"dotnet new webapi" generates a complete starter ASP.NET Core Web API project, including a sample controller, Swagger configuration, and a Program.cs wired up to run it — the same starting point every project in this program, including the Week 5 capstone, will build from.

4.2 The Minimal Hosting Model

Program.cs is where the application is assembled: a WebApplicationBuilder configures services (the dependency injection container), and the resulting WebApplication configures the middleware pipeline before calling Run(). Modern ASP.NET Core collapses what used to be split across Program.cs and Startup.cs into this single, more readable file.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

var app = builder.Build();
app.UseHttpsRedirection();
app.MapControllers();
app.Run();
```

4.3 Controllers vs. Minimal APIs

A Controller groups related endpoints as methods on a class, decorated with route and HTTP verb attributes — familiar, well-suited to larger APIs with many related endpoints sharing common concerns. A Minimal API defines an endpoint as a single lambda mapped directly in Program.cs ("app.MapGet("/users", ...)") — less ceremony for small or simple APIs, but can get unwieldy as an API grows past a handful of endpoints. This program's capstone projects use Controllers as the default, with Minimal APIs introduced today for comparison.

4.4 Routes, Route Parameters, and HTTP Verbs

A route template like "/api/users/{id}" captures a segment of the URL as a parameter bound automatically to a matching method parameter. The HTTP verb (GET, POST, PUT, DELETE) signals intent, not just a URL variation — GET for reading without side effects, POST for creating, PUT for replacing, DELETE for removing — and REST convention expects each verb to be used only for its intended purpose.

A GET endpoint that modifies data breaks a fundamental REST assumption: that GET requests are safe to retry, cache, or prefetch without side effects. Keep verb semantics honest from the very first endpoint.

Hands-On Lab: Scaffold the API & Build First Endpoints

1. Run "dotnet new webapi -o MyFirstApi" and confirm it runs with "dotnet run", checking the Swagger UI in a browser.



2. Add a Controller with a GET endpoint returning a hardcoded list of items from your domain model.
3. Add a GET endpoint with a route parameter that returns a single item by ID.
4. Add the same two endpoints again as Minimal APIs directly in Program.cs, and compare the two approaches.
5. Test all 4 endpoints in Postman and save them as a collection.

Tools Used

.NET SDK • ASP.NET Core • Postman

Day 5 — Middleware Pipeline & Dependency Injection; Week 2 Synthesis — 8 hours

Learning Objectives

- Explain the middleware pipeline and how request execution order is determined.
- Register services with the built-in dependency injection container using the correct lifetime.
- Inject a service into a controller via constructor injection.

Key Topics

- The middleware pipeline
- Built-in middleware and ordering
- Dependency injection and service lifetimes
- Constructor injection in practice

Lesson Content

5.1 The Middleware Pipeline

Every incoming request passes through a chain of middleware components in the exact order they're registered in Program.cs — each middleware can inspect or modify the request, pass it to the next component, and then act again on the way back out. This is the mechanism behind cross-cutting concerns like authentication, logging, and error handling that need to apply to every request without being repeated in every individual endpoint.

5.2 Built-in Middleware and Ordering

UseHttpsRedirection, UseRouting, UseAuthentication, UseAuthorization, and MapControllers are common built-in middleware, and their order matters directly — authorization middleware registered before authentication would have nothing to check against, since the user wouldn't have been identified yet. Getting this order wrong is a common source of confusing, hard-to-diagnose bugs for developers new to ASP.NET Core.

5.3 Dependency Injection and Service Lifetimes

Registering a service with AddTransient creates a new instance every time it's requested; AddScoped creates one instance per HTTP request, shared across everything that needs it within that request; AddSingleton creates exactly one instance for the entire application's lifetime. Choosing the wrong lifetime — registering a database context as Singleton, for instance — causes real bugs from shared, unintentionally leaked state across unrelated requests.

```
builder.Services.AddScoped<IOrderService, OrderService>();

public class OrdersController : ControllerBase
{
    private readonly IOrderService _orderService;
    public OrdersController(IOrderService orderService)
        => _orderService = orderService;
}
```



5.4 Constructor Injection in Practice

A controller declares its dependencies as constructor parameters, typed as interfaces rather than concrete classes wherever possible — the DI container resolves and supplies the actual implementation automatically at request time. This is the same interface-based polymorphism from Week 1's OOP lesson, now doing real structural work: the controller depends on "IOrderService," never on a specific implementation, which is exactly what makes it possible to swap in a test double during unit testing in Week 5.

A service that reaches out and constructs its own dependencies internally ("new OrderRepository()" inside a class) instead of receiving them through its constructor defeats dependency injection entirely, even in a project that otherwise uses DI everywhere else. Dependencies should always be received, not self-constructed.

Hands-On Lab: Middleware & Dependency Injection

1. Write a small custom middleware that logs each request's method and path to the console, and register it in Program.cs.
2. Deliberately place it in the wrong pipeline order once, observe the effect, then correct the ordering.
3. Create an interface and a service class implementing it, and register it with the appropriate lifetime.
4. Inject the service into your Day 4 controller via constructor injection and use it in an endpoint.
5. Assemble a Week 2 summary in Notion: generics/LINQ/async exercises, the scaffolded API, and a link to your GitHub repository — ready for the mentor check-in.

Tools Used

.NET SDK • ASP.NET Core • Postman • Notion

5. Week 2 Deliverables

- A generic repository class with appropriate constraints, committed to GitHub.
- LINQ exercises demonstrating grouping, joining, flattening, and deferred execution.
- An async/concurrency demo using Task.WhenAll and a cancellation token.
- A scaffolded ASP.NET Core Web API with at least 4 endpoints, built with both Controllers and Minimal APIs.
- Custom middleware and at least one DI-registered service injected into a controller.
- A Week 2 summary document in Notion, ready for the mentor check-in.

6. Week 2 Evaluation Criteria

Scored by the assigned mentor at the Week 2 check-in using the criteria below (extracted from the program's 100-point internal rubric).



Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Understanding of core concepts	Partial understanding, often confused	Explains most decisions clearly	Deep understanding, can teach it to others
C# & .NET usage	Basic generics and LINQ only	Interfaces, generics, async/await used correctly	Nullable reference types, advanced LINQ, idiomatic C# throughout
API design & REST principles	Basic routes, wrong HTTP verbs/codes	RESTful, correct status codes	Fully RESTful, consistent conventions across Controllers and Minimal APIs
Code quality & structure	Basic structure, inconsistencies	Clean, readable, consistent patterns	Production-grade, modular, well-organised
Problem-solving & debugging	Struggles to resolve errors independently	Resolves most errors with minimal guidance	Debugs independently, explains root causes clearly
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

7. Technical Stack — Week 2

Category	Tools
Language & Runtime	.NET SDK, C#
Web Framework	ASP.NET Core (Controllers, Minimal APIs, middleware, dependency injection)
API Testing	Postman
Version Control	Git, GitHub
Documentation	Notion (Week 2 summary)

8. Notes & Best Practices

Every ASP.NET Core project this program builds from here forward runs through the exact pipeline set up this week: routing, middleware, and dependency injection. Getting comfortable with how a request moves through that pipeline now makes Week 3's REST API design and Entity Framework Core schema work land on solid ground — by the end of next week, these first hardcoded endpoints will be backed by a real, persisted SQL Server database instead of an in-memory list.